

Documentación final del proyecto

PipelineDesk CRM

Aplicación CRM Pipeline para centros de formación



PipelineDesk

SERGIO PAVÓN DÍAZ

2º DAW · IES Cristóbal de Monroy

Curso 2025-2026



IES Cristóbal de Monroy
Alcalá de Guadaira

Índice

Contenido

Índice	2
1. Introducción	4
1.1 Alcance del documento	4
1.2 Resumen ejecutivo del sistema	4
1.3 Material gráfico de la documentación previa	4
2. Estudio de la competencia	7
2.1 Clasificación de empresas del sector	7
2.2 Necesidades reales del mercado objetivo	7
2.3 Comparativa de soluciones existentes	7
2.4 Conclusión del estudio	8
3. Requisitos del sistema	9
3.0 Funcionalidad general del proyecto	9
3.1 Requisitos de información	9
3.2 Requisitos funcionales	9
3.3 Casos de uso	10
CU-01 · Autenticarse en la aplicación	10
CU-02 · Registrar lead desde formulario web	11
CU-03 · Crear lead interno	11
CU-04 · Cambiar estado desde el panel	11
CU-05 · Registrar actividad en la ficha del lead	11
CU-06 · Mover lead en el Kanban	12
CU-07 · Gestionar tarea	12
CU-08 · Consultar el dashboard	12
4. Modelo de datos	13
4.1 Modelo conceptual	13
4.1.1 Descripción de entidades	13
4.1.2 Descripción de interrelaciones	13
4.2 Modelo lógico de datos	14
4.3 Consultas y vistas	15

5. Interfaz de usuario.....	16
5.1 Definición de pantallas	16
5.1.1 Bocetos y capturas heredadas de la documentación previa.....	16
5.2 Mapa de navegación	22
6. Arquitectura del sistema	24
6.1 Persistencia de datos.....	24
6.2 Procesamiento de información	24
6.3 Interfaz de usuario.....	25
6.3.1 Estructura final del proyecto	27
7. Desarrollo del proyecto del sistema.....	32
7.1 Fase 0 · Base del proyecto	32
7.2 Fase 1 · Usuarios, login y sesiones.....	32
7.3 Fase 2 · Leads y panel comercial	33
7.4 Fase 3 · Detalle del lead, notas e histórico	33
7.5 Fase 4 · Pipeline Kanban	33
7.6 Fase 5 · Tareas y objeciones	34
7.7 Fase 6 · Dashboard y análisis	34
7.8 Fase 7 · Mejora visual, modo oscuro y objetivos	34
8. Información para el cliente	36
8.1 Requisitos físicos y de hardware	36
8.2 Requisitos software	36
8.3 Manual de instalación, configuración y administración.....	36
8.4 Manual de usuario.....	37
9. Bibliografía y fuentes de información	38

1. Introducción

PipelineDesk CRM es una aplicación web desarrollada para gestionar el ciclo comercial de un centro de formación, desde la captación inicial de una persona interesada hasta su conversión en alumno o la pérdida de la oportunidad. El documento presente consolida la memoria funcional y técnica del proyecto a partir de la documentación previa entregada durante el curso y de la revisión de la versión definitiva del código fuente incluida en el ZIP final.

El proyecto se sitúa en el marco del Proyecto Final de 2º DAW y aplica conocimientos de programación en PHP, bases de datos relacionales, arquitectura MVC, maquetación web, JavaScript y mejora progresiva de la interfaz con Tailwind CSS.

La finalidad de este documento es doble: por un lado, servir como memoria técnica y funcional del sistema; por otro, actuar como guía de referencia para comprender su alcance real, su arquitectura y la forma en la que debe instalarse, administrarse y utilizarse.

1.1 Alcance del documento

- Describe el contexto del proyecto y el problema que resuelve.
- Actualiza los requisitos iniciales a la realidad del proyecto implementado.
- Recoge el modelo de datos final y las tablas realmente utilizadas.
- Documenta la interfaz de usuario actual, el flujo de navegación y los módulos disponibles.
- Resume el desarrollo por fases, las pruebas realizadas y el estado final del producto.
- Ofrece manuales de instalación, administración y uso para un entorno local con XAMPP.

1.2 Resumen ejecutivo del sistema

La versión final de PipelineDesk permite autenticación de usuarios, captación pública de leads mediante formulario web, alta manual de leads desde zona privada, panel comercial agrupado por etapas, tablero Kanban, ficha detallada del lead con notas e histórico, módulo de tareas con objeciones y un dashboard analítico con indicadores comerciales, objetivos del mes y notificaciones enriquecidas. La arquitectura se apoya en PHP MVC y MySQL, con JavaScript para interacciones de cliente y Tailwind CSS como refuerzo visual en la fase final.

1.3 Material gráfico de la documentación previa

A continuación se incorporan diagramas e imágenes incluidas en la documentación inicial del proyecto, utilizadas como punto de partida de la propuesta funcional.

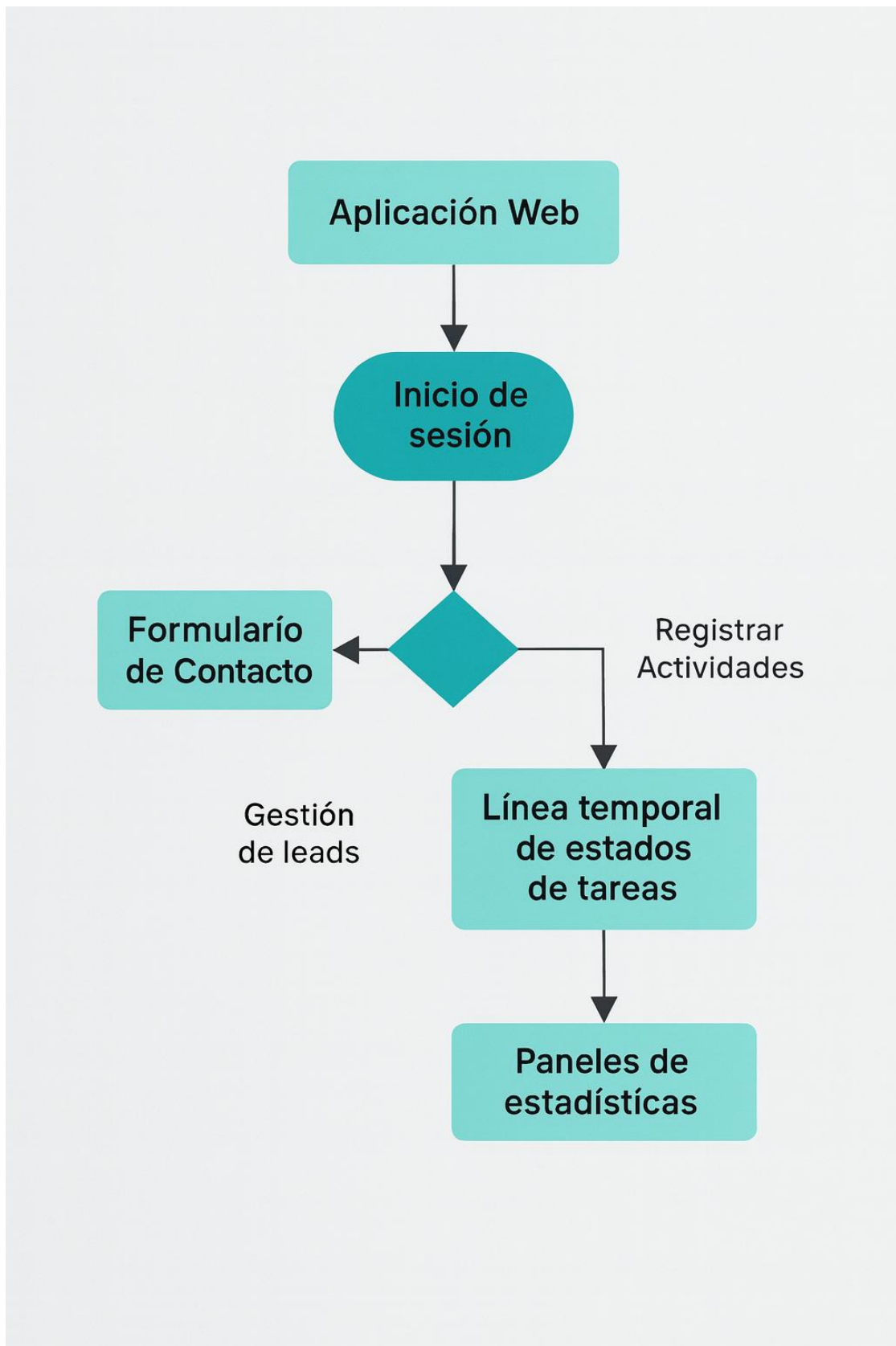


Figura 1. Diagrama inicial de los principales flujos de la aplicación.



Figura 2. Referencia visual de competencia utilizada en el planteamiento inicial.

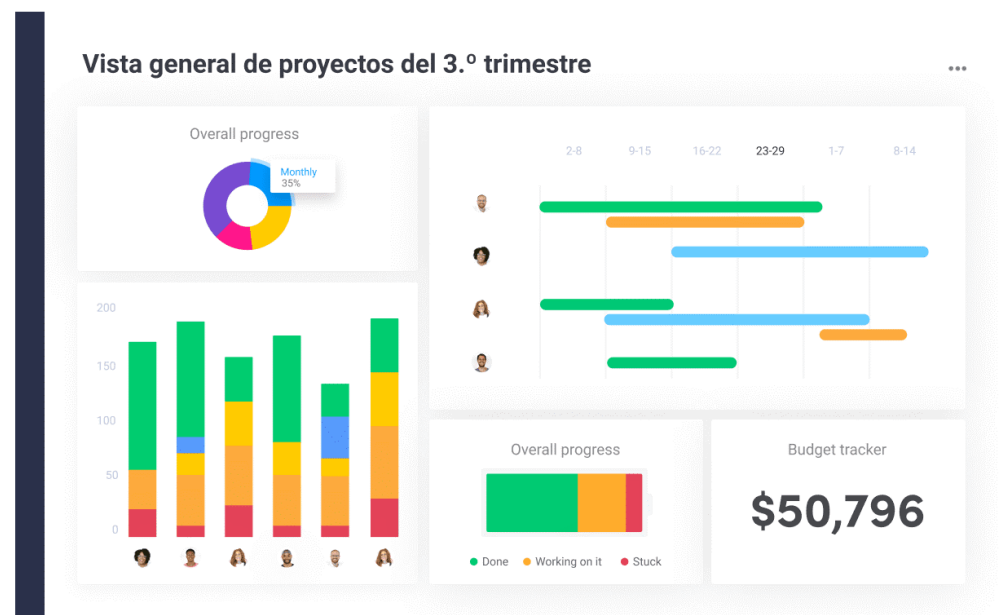


Figura 3. Segunda referencia visual utilizada durante el diseño inicial.

2. Estudio de la competencia

El análisis de competencia se realizó para posicionar PipelineDesk dentro del mercado de herramientas CRM y detectar un hueco específico: soluciones ligeras, comprensibles y adaptadas a academias o centros de formación. Frente a plataformas generalistas o complejas, PipelineDesk se enfoca en captación, seguimiento comercial, embudo de ventas y operativa diaria del vendedor.

2.1 Clasificación de empresas del sector

Grupo	Descripción	Ejemplos	Lectura para PipelineDesk
CRM enterprise	Plataformas muy completas, escalables y con ecosistemas de integración amplios.	Salesforce	Sobredimensionadas para una academia pequeña o mediana.
CRM para pymes	Herramientas más económicas, centradas en ventas y marketing.	HubSpot CRM, Zoho CRM, Efficacy	Competidores directos por sencillez y coste.
CRM integrados en ERP	Módulos CRM conectados con contabilidad, RRHH o facturación.	Odoo CRM	Útiles para organizaciones que priorizan un todo-en-uno.
CRM verticales	Productos orientados a un sector con flujos concretos.	Aplicaciones educativas especializadas	Espacio donde PipelineDesk aporta mayor valor.

2.2 Necesidades reales del mercado objetivo

- Captación automática de leads desde formularios web o campañas.
- Clasificación rápida por curso o servicio de interés.
- Seguimiento comercial sencillo: llamadas, emails, notas y tareas.
- Pipeline visual para saber en qué etapa se encuentra cada oportunidad.
- Análisis básico del rendimiento comercial sin necesidad de herramientas complejas.
- Coste reducido y curva de aprendizaje baja para academias y pequeños equipos.

2.3 Comparativa de soluciones existentes

Aplicación	Fortalezas	Debilidades detectadas	Oportunidad para PipelineDesk
Salesforce	Muy potente, ecosistema amplio, automatización avanzada.	Coste alto y complejidad excesiva para una academia.	Ofrecer un CRM sencillo, vertical y rápido de implantar.
HubSpot CRM	Interfaz clara, plan gratuito atractivo, buena experiencia de usuario.	No está especializado en educación y muchas funciones clave son de pago.	Adaptación del pipeline al proceso de matriculación.
Zoho CRM	Buena relación calidad/precio, alto nivel de personalización.	Necesita configuración inicial y curva de aprendizaje.	Plantillas predefinidas y flujo listo para usar.

Efficy CRM	Solución europea y personalizable.	Menor difusión y enfoque más corporativo.	Propuesta más directa para centros formativos.
Odoo CRM	Potente si se quiere suite completa.	No es un CRM puro y puede ser excesivo para la captación comercial.	Producto centrado en ventas educativas, no en ERP.

2.4 Conclusión del estudio

PipelineDesk se justifica como una solución vertical, académica y orientada a procesos concretos: captación, seguimiento y cierre comercial en centros de formación. No compite en amplitud funcional con los grandes CRM, sino en adecuación, simplicidad y control del flujo real de una academia.

Esta conclusión se mantiene vigente al revisar el ZIP final: la aplicación se ha implementado justo sobre esos ejes diferenciales —pipeline, seguimiento, tareas, objeciones, dashboard y objetivos— y no como un ERP generalista.

Como material gráfico heredado de la documentación previa, se incorporan las portadas de los documentos de análisis de competencia y casos de uso.

Figura 4. Portada del documento de competencia y casos de uso.

3. Requisitos del sistema

En esta sección se describen los requisitos del sistema según la versión final del proyecto. Para actualizar la documentación previa al estado real del código, se distinguen los requisitos que están implementados, los que lo están de forma parcial y los que se han dejado fuera del alcance.

3.0 Funcionalidad general del proyecto

PipelineDesk es un CRM web que organiza el trabajo comercial en torno a leads, etapas del embudo, actividad registrada, tareas y métricas de seguimiento. La aplicación dispone de parte pública (portada y formulario de contacto) y parte privada (panel, pipeline, detalle de leads, tareas, listado, dashboard y políticas de privacidad).

3.1 Requisitos de información

Información gestionada	Descripción en la versión final
Usuarios	La aplicación maneja usuarios internos con nombre, email, contraseña cifrada, rol y estado activo. Los roles reales implementados son admin y ventas.
Leads	Cada lead almacena nombre, estado, responsable, servicio de interés, indicaciones, score, email, teléfono, valor económico, último contacto, prioridad, origen y fechas de creación/actualización.
Notas/actividades	Las actividades registradas en la ficha del lead se guardan como notas con tipo de actividad, contenido, usuario y fecha.
Historial	Cada lead conserva un historial de eventos relevantes: alta, notas y cambios de estado.
Tareas	Las tareas están ligadas a un lead y a dos usuarios (creador y asignado), con tipo de actividad, fecha final, estado y descripción.
Objeciones	Las objeciones se modelan dentro de las tareas mediante tipo_bloqueo y solucion_bloqueo.
Objetivos y notificaciones	El modelo final añade tablas de soporte para objetivos_mensuales y notificaciones_app, creadas automáticamente por LeadModel si no existen.
Filtros y analítica	Dashboard y listado manejan filtros por usuario, fecha, servicio, estado y origen para obtener vistas resumidas del sistema.

No forman parte de la versión final la gestión de documentos adjuntos, recuperación de contraseña, roles avanzados adicionales o un catálogo administrable de cursos. Estas ideas estaban presentes en documentación previa, pero no aparecen en el ZIP final.

3.2 Requisitos funcionales

Código	Requisito funcional	Estado final
RF-01	Inicio y cierre de sesión con control de acceso por rutas privadas.	Implementado

RF-02	Alta automática de leads desde formulario público.	Implementado
RF-03	Alta manual de leads desde zona privada.	Implementado
RF-04	Visualización de leads agrupados por estado en panel.	Implementado
RF-05	Cambio de estado de un lead desde panel.	Implementado
RF-06	Cambio de estado visual mediante tablero Kanban.	Implementado
RF-07	Consulta detallada de un lead.	Implementado
RF-08	Alta de actividades/notas sobre un lead.	Implementado
RF-09	Edición y borrado de leads según permisos.	Implementado (borrado solo admin)
RF-10	Gestión operativa de tareas.	Implementado
RF-11	Creación automática de tareas de objeción cuando el lead entra en Objeciones.	Implementado
RF-12	Dashboard comercial con filtros e indicadores.	Implementado
RF-13	Listado filtrable de leads.	Implementado
RF-14	Modo oscuro y notificaciones toast apilables.	Implementado
RF-15	Edición de usuarios desde interfaz.	No implementado
RF-16	Recuperación de contraseña.	No implementado
RF-17	Adjuntar documentos al lead.	No implementado
RF-18	Moderación de contenidos / exportación de informes.	No implementado

3.3 Casos de uso

Se recogen a continuación los casos de uso más relevantes de la versión final. Se han reescrito para ajustarlos al alcance real implementado.

CU-01 · Autenticarse en la aplicación

Campo	Detalle
Actor principal	Usuario interno (admin o ventas)
Precondiciones	Usuario activo creado en base de datos.
Flujo principal	1. Accede a /login. 2. Introduce email y contraseña. 3. El sistema valida datos, busca el usuario y verifica password_hash. 4. Si todo es correcto, guarda el usuario en sesión y redirige a /panel.
Postcondición	El usuario inicia sesión y obtiene acceso a rutas privadas.

CU-02 · Registrar lead desde formulario web

Campo	Detalle
Actor principal	Visitante externo
Precondiciones	La página de contacto está disponible.
Flujo principal	1. Completa nombre, email, teléfono, servicio e indicaciones. 2. El sistema valida campos obligatorios. 3. Inserta el lead con origen formulario_web. 4. Registra historial de alta y muestra mensaje de confirmación.
Postcondición	El lead queda creado con estado inicial y disponible en la zona privada.

CU-03 · Crear lead interno

Campo	Detalle
Actor principal	Usuario autenticado
Precondiciones	Sesión iniciada.
Flujo principal	1. Accede a /leads/nuevo. 2. Rellena el formulario interno. 3. El sistema valida datos y guarda el registro. 4. Si el estado inicial es Objeciones, crea la tarea automática correspondiente.
Postcondición	El lead queda dado de alta y visible en panel/pipeline.

CU-04 · Cambiar estado desde el panel

Campo	Detalle
Actor principal	Usuario autenticado
Precondiciones	Lead existente.
Flujo principal	1. Accede al panel. 2. Selecciona otro estado en el selector visual. 3. El sistema envía POST a /leads/cambiar-estado/{id}. 4. Se actualiza la base de datos y se registra el histórico.
Postcondición	El lead se reagrupa en su nueva columna/tabla.

CU-05 · Registrar actividad en la ficha del lead

Campo	Detalle
Actor principal	Usuario autenticado
Precondiciones	Lead existente y sesión iniciada.
Flujo principal	1. Abre el detalle del lead. 2. Rellena tipo de actividad y contenido. 3. El sistema guarda la nota, actualiza ultimo_contacto y crea histórico. 4. Si el estado era Nuevo Lead o Contactado, avanza automáticamente a la siguiente etapa.
Postcondición	La actividad queda registrada y el lead mantiene

	trazabilidad comercial.
--	-------------------------

CU-06 · Mover lead en el Kanban

Campo	Detalle
Actor principal	Usuario autenticado
Precondiciones	Lead visible en la vista pipeline.
Flujo principal	1. Arrastra una tarjeta a otra columna. 2. JavaScript envía la actualización al endpoint del pipeline. 3. El backend valida el nuevo estado. 4. Si la operación es correcta, persiste el cambio e informa al cliente.
Postcondición	El lead cambia de etapa sin salir del tablero.

CU-07 · Gestionar tarea

Campo	Detalle
Actor principal	Usuario autenticado
Precondiciones	Existe un lead y, en su caso, permisos sobre la tarea.
Flujo principal	1. Accede a /tareas. 2. Crea, edita, cambia estado o elimina una tarea. 3. Si la tarea es de objeciones, define bloqueo y solución. 4. El sistema actualiza la tarea y el último contacto del lead.
Postcondición	La tarea queda sincronizada con la operativa del lead.

CU-08 · Consultar el dashboard

Campo	Detalle
Actor principal	Usuario autenticado
Precondiciones	Sesión activa.
Flujo principal	1. Accede a /dashboard. 2. Aplica filtros opcionales. 3. El sistema calcula indicadores agregados desde modelos LeadModel y TareaModel. 4. Muestra objetivos del mes, métricas de embudo, productividad y seguimientos.
Postcondición	El usuario obtiene una vista analítica del estado comercial.

4. Modelo de datos

El modelo de datos final del proyecto se ha ajustado a la implementación real en MySQL. A diferencia de documentos previos basados en una versión conceptual más amplia, la base de datos definitiva se centra en siete tablas operativas: usuarios, leads, notas_lead, historial_lead, tareas_lead y dos tablas de soporte creadas dinámicamente por el modelo de leads: objetivos_mensuales y notificaciones_app.

4.1 Modelo conceptual

El sistema se estructura alrededor de la entidad lead. Un lead pertenece opcionalmente a un responsable, puede tener múltiples notas y múltiples eventos de histórico, y además puede originar tareas operativas ligadas a usuarios. En fases posteriores se añadió el concepto de objetivo mensual y de notificación enriquecida.

4.1.1 Descripción de entidades

Entidad	Función
usuarios	Almacena credenciales, rol y estado del personal interno que accede al CRM.
leads	Entidad central del sistema; representa a un alumno potencial o oportunidad comercial.
notas_lead	Registra actividades comerciales realizadas sobre un lead.
historial_lead	Mantiene la trazabilidad de altas, notas y cambios de estado.
tareas_lead	Gestiona seguimientos, tareas y objeciones asociadas a leads.
objetivos_mensuales	Tabla de soporte para objetivos comerciales mensuales; se crea automáticamente si no existe.
notificaciones_app	Tabla de soporte para notificaciones internas, incluida la felicitación por lead ganado.

4.1.2 Descripción de interrelaciones

Interrelación	Descripción
usuarios 1:N leads	Un usuario puede ser responsable de muchos leads; un lead tiene un responsable_id opcional.
leads 1:N notas_lead	Cada lead puede tener múltiples actividades comerciales registradas.
usuarios 1:N notas_lead	Cada nota la crea un único usuario autenticado.
leads 1:N historial_lead	El histórico almacena múltiples eventos por lead.
usuarios 1:N historial_lead	Un usuario puede generar cambios de estado o notas sobre múltiples leads.
leads 1:N tareas_lead	Cada tarea pertenece a un lead concreto en la versión final.
usuarios 1:N tareas_lead (creador)	usuario_creador_id registra quién creó la tarea.
usuarios 1:N tareas_lead (asignado)	usuario_asignado_id registra quién debe ejecutarla.
usuarios 1:N notificaciones_app	Cada usuario puede recibir varias notificaciones internas.
leads 1:N notificaciones_app	Las notificaciones pueden enlazarse con un lead concreto.

Se incorpora a continuación el diagrama entidad-relación heredado de la documentación previa. Aunque la versión final se ha ajustado al stack PHP y a nombres de tablas definitivos en castellano, la figura sigue siendo útil como apoyo visual para comprender las relaciones generales del sistema.

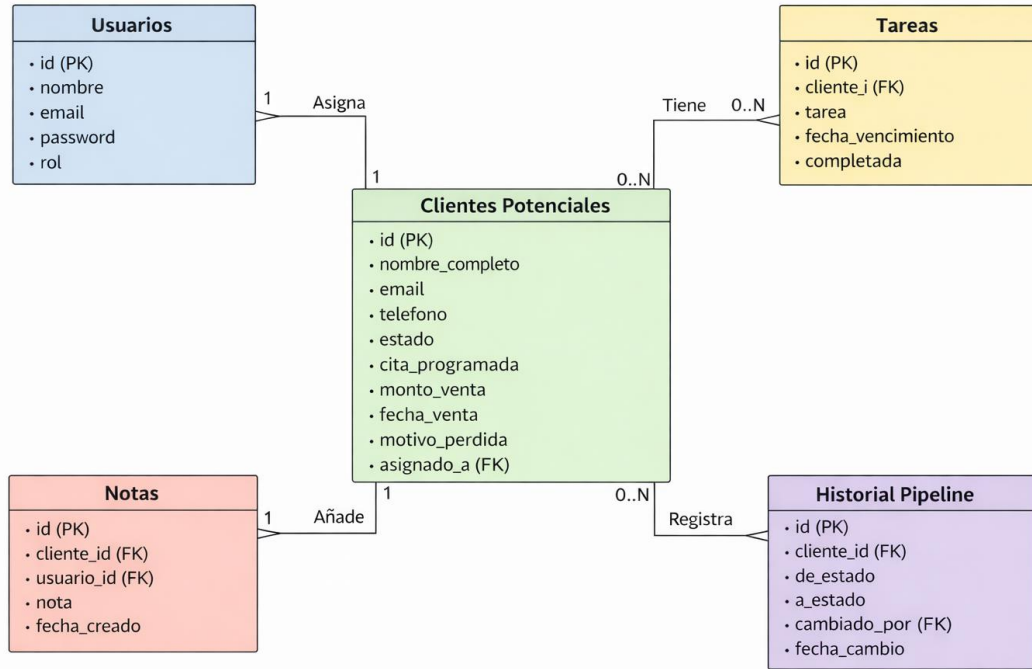


Figura 6. Diagrama entidad-relación procedente de la documentación previa.

4.2 Modelo lógico de datos

El script principal de instalación es database/migrations/pipeline_crm.sql. A continuación se resumen las tablas efectivamente usadas por la aplicación y los campos más importantes.

Tabla	Campos clave
usuarios	id (PK), nombre, email UNIQUE, password_hash, rol, activo, created_at
leads	id (PK), lead_nombre, estado, responsable_id (FK), servicios, indicaciones, lead_score, email, telefono, valor, ultimo_contacto, prioridad, origen, created_at, updated_at
notas_lead	id (PK), lead_id (FK), usuario_id (FK), tipo_actividad, contenido, created_at
historial_lead	id (PK), lead_id (FK), usuario_id (FK nullable), tipo_evento, titulo, descripcion, estado_anterior, estado_nuevo, created_at
tareas_lead	id (PK), lead_id (FK), usuario_creador_id (FK), usuario_asignado_id (FK), tipo_actividad, tipo_bloqueo, solucion_bloqueo, descripcion, fecha_final, estado, leida_asignado, created_at,

	updated_at
objetivos_mensuales	id (PK), anio, mes, objetivo_leads, usuario_id (FK nullable), created_at, updated_at
notificaciones_app	id (PK), usuario_id (FK), usuario_origen_id (FK nullable), lead_id (FK nullable), tipo, titulo, mensaje, enlace, leida, created_at

4.3 Consultas y vistas

En la versión final no se han definido vistas SQL persistentes para explotación analítica. El análisis del sistema se resuelve directamente en los modelos mediante consultas SELECT preparadas y agregaciones. El archivo database/views/50_lead.sql se utiliza como carga de datos de prueba y no como definición de vistas de MySQL.

Uso	Consulta dominante	Método responsable
Autenticación	SELECT ... FROM usuarios WHERE email = ? LIMIT 1	LoginModel::findByEmail
Panel por estados	SELECT leads + LEFT JOIN usuarios ORDER BY created_at	LeadModel::obtenerAgrupadosPorEstado
Detalle del lead	SELECT lead por id; SELECT notas por lead; SELECT historial por lead	LeadModel::findById / getNotasByLead / getHistorialByLead
Cambio de estado	UPDATE leads SET estado = ?, lead_score = CASE ...	LeadModel::updateEstado
Resumen dashboard	SELECT COUNT, SUM, CASE WHEN por estado	LeadModel::getDashboardResumenGeneral
Embudo comercial	SELECT estado, COUNT, SUM(valor), AVG(...) GROUP BY estado	LeadModel::getResumenPipeline
Leads sin contacto	SELECT ... WHERE ultimo_contacto IS NULL	LeadModel::getLeadsSinContacto
Listado filtrado	SELECT leads + filtros dinámicos por fecha/estado/origen/usuario	LeadModel::getListadoFiltrado
Tareas listadas	SELECT tareas + INNER JOIN leads/usuarios + ORDER BY CASE	TareaModel::getTareasListado
Tareas de dashboard	SELECT agregados por estado, retrasos y objeciones	TareaModel::*Dashboard*

Los patrones SQL que aparecen de forma recurrente y que conviene conocer para comprender el proyecto son: SELECT con filtros parametrizados, INSERT para altas, UPDATE para cambios de estado o edición, DELETE controlado por permisos, INNER JOIN y LEFT JOIN para unir tablas, GROUP BY para estadísticas, CASE WHEN para cálculos condicionales y EXISTS para verificar relaciones.

5. Interfaz de usuario

La interfaz de usuario se ha desarrollado con HTML, CSS y JavaScript. A partir de la fase 7 se ha reforzado visualmente con Tailwind CSS sin sustituir el CSS previo, lo que permite mantener la estética propia del proyecto y añadir un modo oscuro coherente.

5.1 Definición de pantallas

Pantalla	Archivo de vista	Finalidad
Portada pública	index_view.php	Presenta la marca, explica el producto y dirige al usuario al login o al formulario de contacto.
Login	login_view.php	Autenticación de usuarios internos.
Contacto público	contacto_view.php	Captación externa de leads desde la web.
Panel	panel_view.php	Vista principal de trabajo: leads agrupados por estado, selector visual, acceso rápido a detalle.
Pipeline	kanban_view.php	Tablero Kanban para mover leads entre etapas con drag and drop.
Nuevo lead	lead_create_view.php	Formulario interno de alta manual de leads.
Detalle del lead	detalles_view.php	Ficha completa con datos, actividad, histórico, edición y cambio de estado.
Tareas	tareas_view.php	Módulo operativo de tareas, objeciones, seguimiento y edición inline.
Dashboard	dashboard_view.php	Vista analítica con filtros, objetivos del mes y resúmenes comerciales.
Listado	listado_view.php	Vista de todos los leads con filtros por curso, mes, vendedor y estado.
Política de privacidad	privacy_view.php	Página legal accesible desde el footer.

5.1.1 Bocetos y capturas heredadas de la documentación previa

En la memoria final se incorporan todas las imágenes disponibles en los documentos previos, tanto como material de referencia como para dejar constancia del proceso de diseño.

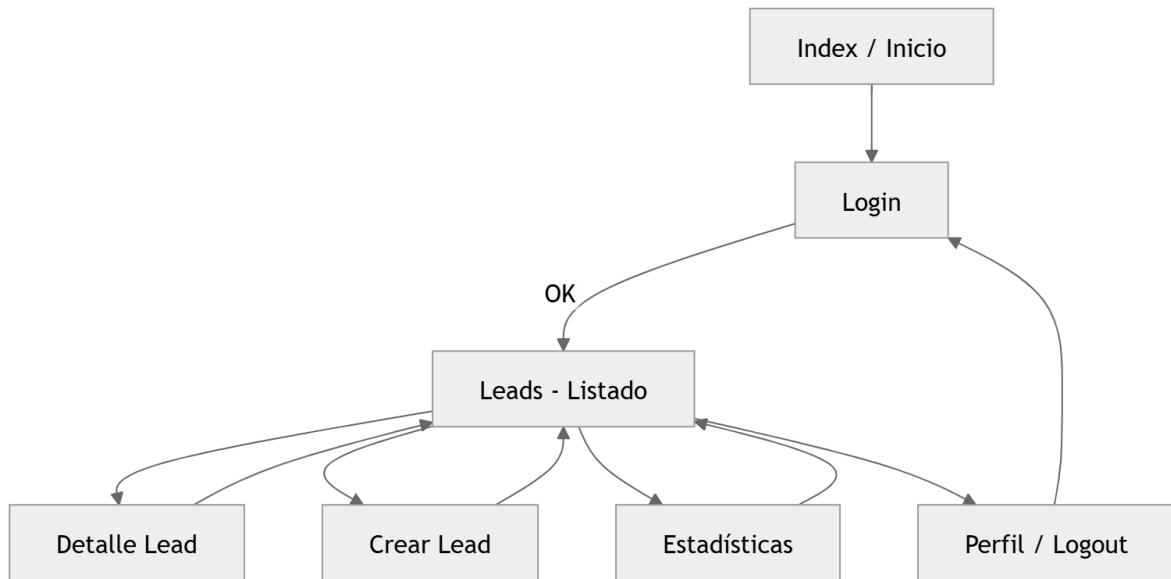


Figura 7. Mapa de navegación del usuario de ventas.

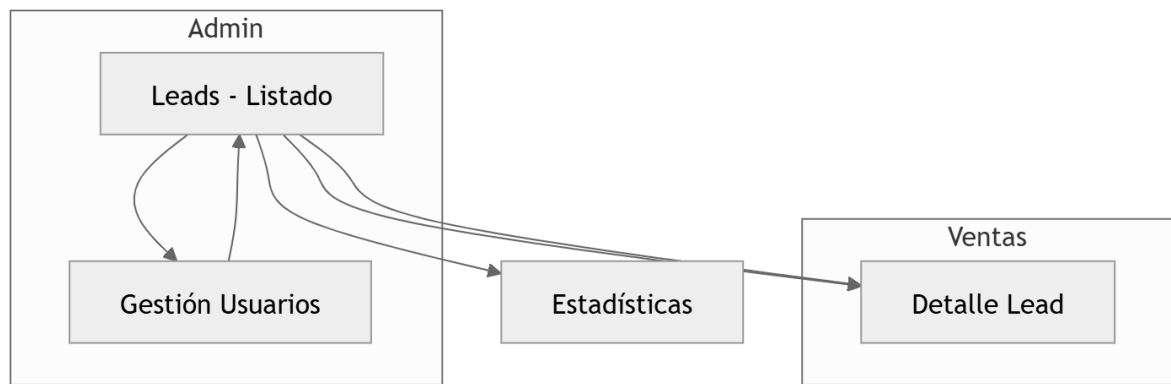


Figura 8. Mapa de navegación del usuario administrador.



Figura 9. Boceto de la pantalla de login.



Figura 10. Boceto del dashboard / home.

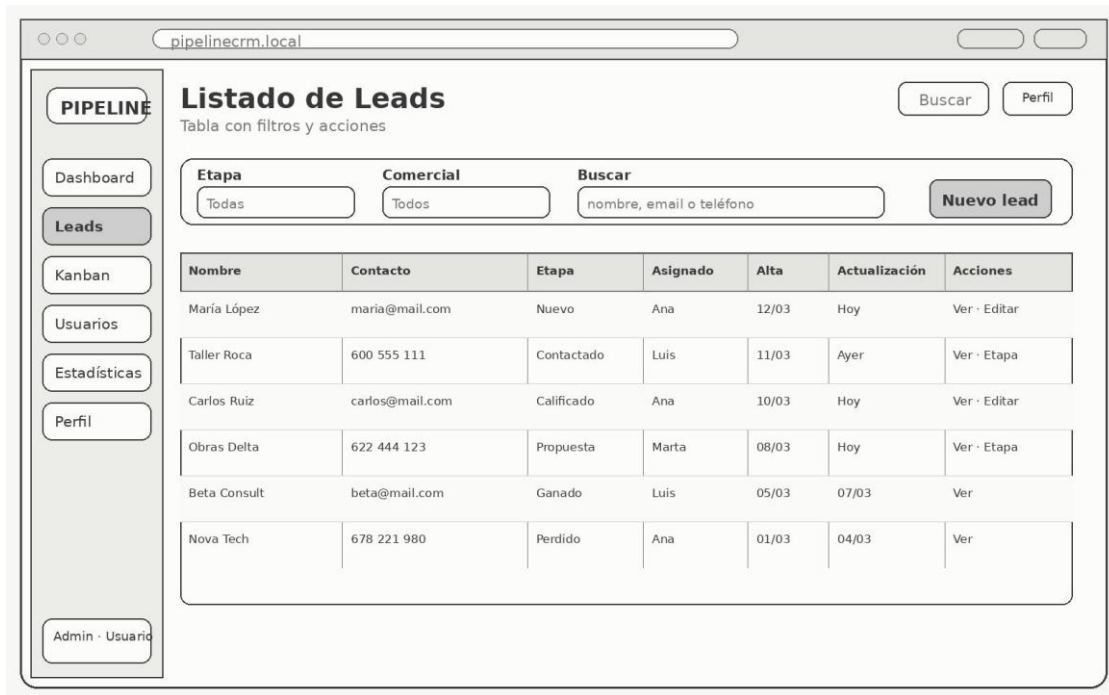


Figura 11. Boceto del listado de leads.

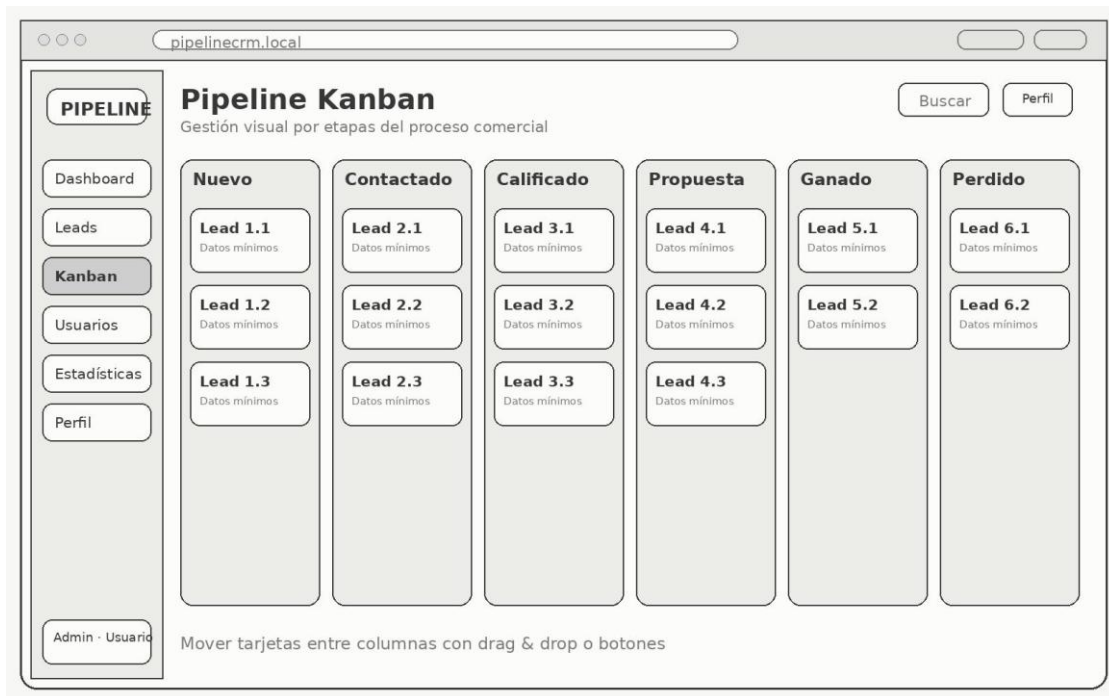


Figura 12. Boceto del tablero Kanban.

pipelinecrm.local

PIPELINE

Dashboard

Leads

Kanban

Usuarios

Estadísticas

Perfil

Admin · Usuario

Nuevo Lead

Formulario de alta

Buscar Perfil

Datos del lead

Nombre

Maria López

Email

maria@empresa.com

Teléfono

600 123 456

Origen

Formulario web

Observaciones iniciales

Interesada en solicitar una demo del servicio. Necesita presupuesto y seguimiento telefónico.

Ayuda

Campos mínimos recomendados según el documento:

- Nombre
- Email
- Teléfono
- Origen
- Observaciones

Guardar **Cancelar**

Figura 13. Boceto de la pantalla de creación de lead.

pipelinecrm.local

PIPELINE

Dashboard

Leads

Kanban

Usuarios

Estadísticas

Perfil

Admin · Usuario

Detalle del Lead

Ficha completa, notas y actividades

Buscar Perfil

Ficha del lead

Nombre

Maria López

Etapa

Calificado

Asignado

Ana Pérez

Origen

Formulario web

Historial

- Nuevo → Contactado
- Contactado → Calificado
- Llamada registrada
- Nota añadida

Notas

Interés alto. Solicita comparativa.

Enviar propuesta esta semana.

Prefiere contacto por email.

Actividades

Registrar

Fecha	Tipo	Descripción	Estado
12/03	Llamada	Primera toma de contacto	Hecha
13/03	Email	Envío de información	Hecha
15/03	Reunión	Demo del producto	Pendiente

Figura 14. Boceto del detalle del lead.

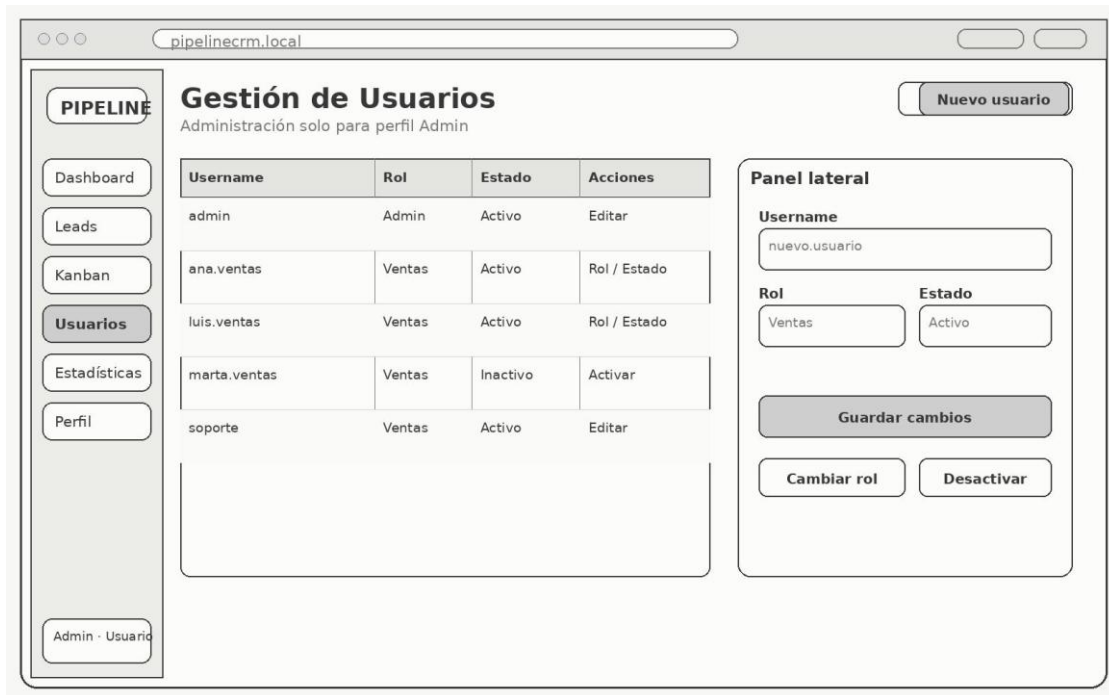


Figura 15. Boceto de la gestión de usuarios propuesta en documentación inicial.

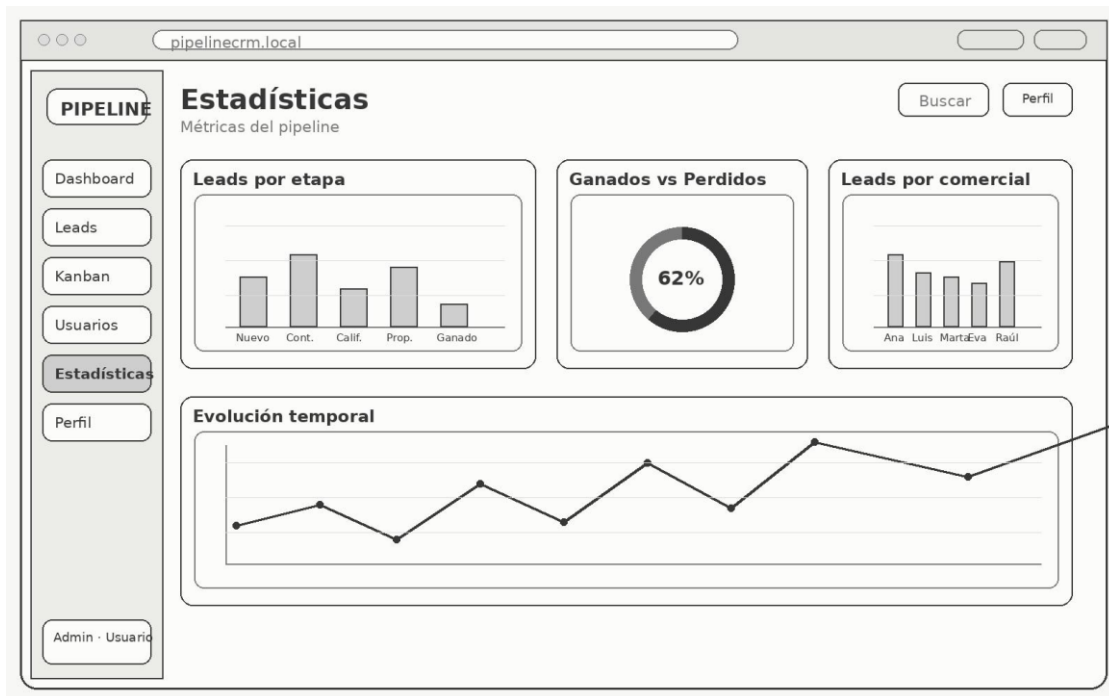


Figura 16. Boceto de la pantalla de estadísticas.

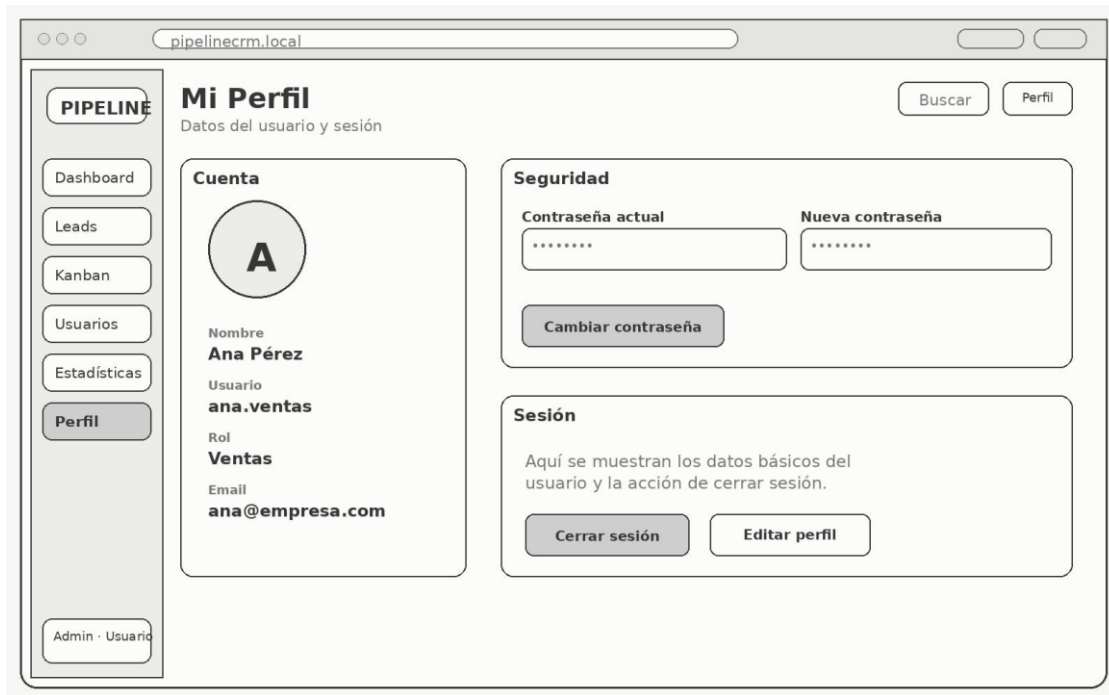


Figura 17. Boceto de perfil / mi cuenta previsto en diseño inicial.

5.2 Mapa de navegación

El flujo de navegación definitivo se organiza en dos áreas: pública y privada.

Zona pública

/ -> portada
/login -> acceso privado
/contacto -> formulario de captación
/politica-privacidad -> página legal

Zona privada

/panel -> panel principal
/pipeline -> kanban del embudo
/dashboard -> análisis y objetivos
/leads/nuevo -> alta interna
/leads/{id} -> detalle del lead
/leads/listado -> listado filtrable
/tareas -> tareas y objeciones
/logout -> cierre de sesión

Desde el punto de vista MVC, cada navegación GET carga una vista y cada acción POST ejecuta una operación de negocio y termina con redirección (patrón POST/Redirect/GET). Esto evita reenvíos accidentales del formulario al recargar la página.

6. Arquitectura del sistema

La arquitectura definitiva del sistema es una arquitectura web clásica basada en PHP con patrón MVC y front controller. La aplicación no utiliza framework completo; en su lugar, implementa sus propios componentes básicos de configuración, enrutado, sesión, acceso a datos y renderizado de vistas.

El flujo técnico general es el siguiente: el navegador solicita una ruta; `public/index.php` arranca Composer, configuración y rutas; `Route` resuelve el controlador; el controlador coordina sesión, validación, modelos y vista; el modelo ejecuta consultas sobre MySQL mediante la clase `Database`; finalmente se devuelve HTML renderizado al navegador.

6.1 Persistencia de datos

La persistencia se apoya en MySQL mediante la clase `lib/Database.php`, que encapsula la conexión y ofrece métodos de consulta y actualización basados en sentencias preparadas de MySQLi.

- Conexión configurada mediante constantes `BD_HOST`, `BD_USER`, `BD_PASS` y `BD_NAME` en `lib/db_credentials.php`.
- Codificación `utf8` para homogeneizar texto y acentos.
- Uso de consultas preparadas en `executeQuery()` y `executeUpdate()` para reducir riesgo de inyección SQL.
- Compatibilidad con transacciones (`beginTransaction`, `commit`, `rollback`), aunque en el proyecto final se usan solo puntualmente.
- Modelo de datos relacional con claves foráneas para mantener integridad.

La persistencia está diseñada para un entorno local con XAMPP. Los scripts de migración incluidos permiten reconstruir la base de datos desde cero y cargar usuarios de prueba.

6.2 Procesamiento de información

El procesamiento de información reside en la capa backend escrita en PHP. La lógica se distribuye entre controllers y models.

Capa	Archivos principales	Responsabilidad
Front controller	<code>public/index.php</code>	Cargar autoload, configuración y rutas.
Router	<code>lib/Route.php</code>	Mapear URI y método HTTP al controlador correspondiente.
Sesión	<code>lib/SessionManager.php</code>	Control de autenticación y mensajes flash/toast.
Controladores	<code>app/controllers/*.php</code>	Validar entradas, coordinar modelos y decidir la vista o redirección.
Modelos	<code>app/models/*.php</code>	Ejecutar SQL y devolver arrays listos para la vista.
Vistas	<code>app/views/**</code>	Renderizar HTML y componentes visuales.

La solución adopta una separación clara: el modelo no pinta HTML, la vista no ejecuta SQL y el controlador no contiene maquetación. Esta organización facilita el aprendizaje, el mantenimiento y la escalabilidad.

6.3 Interfaz de usuario

La interfaz se implementa con plantillas PHP dentro de `app/views`, estilos por vista dentro de `public/css` y scripts específicos dentro de `public/js`. Además, existe un layout común con header, footer, aside lateral, switch de tema y sistema global de toasts.

- CSS base: `public/css/style.css`.
- CSS por vista: `login.css`, `panel.css`, `kanban.css`, `dashboard.css`, `tareas.css`, etc.
- JavaScript específico: `panel.js`, `kanban.js`, `detalles.js`, `tareas.js`, `dashboard.js`, `flash-toast.js` y `theme.js`.
- Tailwind CSS: definido en `tailwind-source.css` y compilado a `tailwind.css` mediante scripts npm.
- Responsive: la navegación privada cambia a menú hamburguesa en tablet y móvil.

Se incluyen a continuación materiales visuales procedentes del documento técnico de arquitectura, útiles para ilustrar la evolución de la interfaz y la estructura general del proyecto.

PipelineDesk · BACKEND

Versión reducida · ideas principales

Raíz	- pipeline-crm/ └ pom.xml # Parent Maven └ README.md # Guía del proyecto └ .env.example # Variables de entorno └ docker-compose.yml # MySQL local
BD	- db/ # Base de datos └ schema.sql # Estructura de tablas └ seed.sql # Datos iniciales
Dominio	- domain/ # Modelo de dominio puro └ lead/ # Lead y etapas └ note/ # Notas └ activity/ # Actividades └ user/ # Usuarios y roles └ valueobjects/ # Validaciones └ common/ # Reglas y excepciones
Aplicación	- application/ # Casos de uso y puertos └ ports/inbound/ # Servicios de entrada └ ports/outbound/ # Repositorios └ usecases/ # Lógica de aplicación └ common/ # Excepciones app
Adaptador	- adapters/ # Entrada y salida └ inbound-rest/ # API REST └ resources/ # Endpoints └ dto/ # Requests y responses └ mappers/ # Conversión REST └ errors/ # Manejo de errores HTTP └ outbound-mysql/ # Persistencia MySQL └ config/ # DataSource y persistencia └ entities/ # Entidades BD └ repositories/ # Repositorios MySQL └ mappers/ # Conversión dominio-BD └ resources/META-INF/ # persistence.xml
Bootstrap	- bootstrap-openliberty/ # Servidor y arranque └ server.xml # Configuración Liberty └ StartupChecks.java # Validaciones de inicio
Docs	docs/ # Documentación técnica

Figura 19. Esquema de evolución del proyecto por fases.

PipelineDesk · FRONTEND

Versión reducida · ideas principales

Páginas	<pre> pipeline-crm/ ├── frontend/ │ ├── index.html │ ├── leads.html │ ├── lead-detalle.html │ ├── usuarios.html │ ├── login.html │ ├── estadisticas.html │ └── dashboard.html </pre>	<pre> # Interfaz web # Entrada principal # Alta y listado # Detalle del lead # Gestión de usuarios # Acceso al sistema # Métricas # Resumen general </pre>
CSS	<pre> ├── assets/ │ ├── css/ │ │ ├── main.css │ │ └── layout.css </pre>	<pre> # Estilos # Estilos generales # Estructura visual </pre>
JS	<pre> ├── dashboard.css ├── js/ │ ├── core/ │ ├── services/ │ └── pages/ </pre>	<pre> # Panel y métricas # Config, API, HTTP, storage, DOM # Leads, notas, actividades, auth, users, stats # Lógica por pantalla </pre>
Imagen	<pre> ├── components/ └── img/ </pre>	<pre> # Navbar, sidebar, modal, table, toast # Logo e iconos </pre>

Figura 20. Segundo esquema evolutivo incluido en la documentación técnica.

6.3.1 Estructura final del proyecto



crm-pipeline/

└─ .env

└─ .env.example

└─ composer.json

└─ package.json

|

└─ app/

| └─ controllers/ -> lógica de entrada por casos de uso

| | └─ Controller.php

| | └─ HomeController.php

| | └─ LeadController.php

| | └─ LoginController.php

| | └─ TareaController.php

| |

| └─ models/ -> consultas y lógica de acceso a datos

| | └─ LeadModel.php

| | └─ LoginModel.php

| | └─ TareaModel.php

| |

| └─ views/ -> vistas home, lead y layouts comunes

| └─ home/

| └─ lead/

| └─ layouts/

|

└─ database/

| └─ migrations/ -> scripts SQL de instalación

- | | ├── 000_create_database.sql
- | | ├── 001_create_usuarios.sql
- | | ├── 002_create_leads.sql
- | | ├── 003_create_contactos.sql
- | | ├── 004_create_actividades.sql
- | | ├── 005_create_tareas.sql
- | | └── pipeline_crm.sql
- | |
- | └── views/ -> cargas de datos de prueba
 - | └── 50_lead.sql
- |
- └── lib/ -> configuración, router, sesiones y base de datos
 - | ├── Database.php
 - | ├── Route.php
 - | ├── SessionManager.php
 - | ├── config.php
 - | └── db_credentials.php
- |
- └── public/
 - | ├── css/ -> hojas de estilo por vista y Tailwind compilado
 - | | ├── contacto.css
 - | | ├── dashboard.css
 - | | ├── detalles.css
 - | | ├── flash-toast.css
 - | | ├── footer.css
 - | | └── index.css

| | └─ kanban.css

| | └─ lead_create.css

| | └─ listado.css

| | └─ login.css

| | └─ panel.css

| | └─ style.css

| | └─ tailwind.css

| | └─ tailwind.min.css

| | └─ tareas.css

| |

| └─ js/ -> scripts de comportamiento

| | └─ app.js

| | └─ dashboard.js

| | └─ detalles.js

| | └─ flash-toast.js

| | └─ index.js

| | └─ kanban.js

| | └─ listado.js

| | └─ panel.js

| | └─ tareas.js

| | └─ theme.js

| |

| └─ img/ -> logotipo, imágenes de usuarios y recursos visuales

| | └─ logo-crm.png

| | └─ logo-crm@2x.png

| | └─ user1.png

| | └─ user2.png

| | └─ user3.png

| |

| └─ index.php -> punto de entrada público

|

└─ routes/

| └─ web.php -> mapa de rutas

|

└─ docs/ -> documentación y README por fases

7. Desarrollo del proyecto del sistema

El desarrollo se ha realizado de forma incremental. Cada fase añadió funcionalidades nuevas y consolidó la base de la anterior. Se resume primero el alcance global y después se detalla cada fase con funcionalidades y plan de pruebas.

Fase	Contenido principal	Estado
Fase 0	Base MVC, router, layouts y estructura del proyecto.	Completada
Fase 1	Usuarios, login, sesiones y panel privado inicial.	Completada
Fase 2	Leads, formulario público e interno, panel por estados.	Completada
Fase 3	Detalle del lead, notas, historial y edición avanzada.	Completada
Fase 4	Kanban interactivo con persistencia asíncrona.	Completada
Fase 5	Tareas, objeciones, productividad y automatización inicial.	Completada
Fase 6	Dashboard analítico y listado filtrable.	Completada
Fase 7	Tailwind progresivo, modo oscuro, toasts y objetivos del mes.	Completada

7.1 Fase 0 · Base del proyecto

Funcionalidades desarrolladas:

- Estructura MVC inicial.
- Front controller en public/index.php.
- Clase Route con soporte para GET, POST y parámetros.
- Layouts comunes y primeros estilos.
- Configuración de constantes y base URL.

Plan de pruebas y comprobaciones:

- Abrir / y comprobar carga de portada.
- Verificar que rutas inexistentes devuelven 404.
- Comprobar que BASE_URL se calcula correctamente en subdirectorio.

7.2 Fase 1 · Usuarios, login y sesiones

Funcionalidades desarrolladas:

- Creación de la tabla usuarios.
- LoginModel para búsqueda por email.
- Validación de credenciales con password_verify.
- Guardado de usuario en sesión sin password_hash.
- Protección de rutas privadas y logout.

Plan de pruebas y comprobaciones:

- Intentar entrar en /panel sin sesión: debe redirigir a /login.
- Iniciar sesión con usuario activo válido.
- Probar contraseña incorrecta.
- Probar usuario inactivo.
- Cerrar sesión y verificar destrucción de sesión.

7.3 Fase 2 · Leads y panel comercial

Funcionalidades desarrolladas:

- Formulario público de contacto.
- Formulario interno de nuevo lead.
- Validación de campos y sticky form.
- Panel con leads agrupados por estado.
- Cambio de estado desde panel.

Plan de pruebas y comprobaciones:

- Enviar formulario público completo.
- Enviar formulario interno con errores y verificar mensajes de campo.
- Cambiar estado desde el panel y comprobar reagrupación.
- Comprobar creación de histórico de alta.

7.4 Fase 3 · Detalle del lead, notas e histórico

Funcionalidades desarrolladas:

- Vista detalle del lead.
- Notas/actividades comerciales.
- Historial de eventos.
- Edición de lead desde la propia ficha.
- Borrado de lead limitado a admin.

Plan de pruebas y comprobaciones:

- Acceder a detalle de un lead existente.
- Registrar actividad y verificar nota e histórico.
- Editar campos y comprobar persistencia.
- Intentar eliminar con usuario ventas y verificar bloqueo.

7.5 Fase 4 · Pipeline Kanban

Funcionalidades desarrolladas:

- Nueva vista Kanban independiente del panel.
- Tarjetas de lead por columnas.
- Drag and drop HTML5.
- Persistencia mediante fetch y endpoint POST.

- Contadores por columna.

Plan de pruebas y comprobaciones:

- Mover lead entre columnas y verificar cambio de estado en base de datos.
- Probar arrastre inválido o mismo estado.
- Comprobar actualización de contadores y valores.

7.6 Fase 5 · Tareas y objeciones

Funcionalidades desarrolladas:

- Módulo Tareas.
- CRUD de tareas.
- Edición inline.
- Creación automática de tarea cuando un lead entra en Objeciones.
- Bloqueo y solución con textos INFO.

Plan de pruebas y comprobaciones:

- Crear tarea manual.
- Editar tarea en línea.
- Cambiar un lead a Objeciones y comprobar tarea automática.
- Verificar filas retrasadas y resumen de cabecera.
- Comprobar permisos admin/ventas.

7.7 Fase 6 · Dashboard y análisis

Funcionalidades desarrolladas:

- Dashboard con filtros.
- Indicadores del embudo.
- Leads sin contacto.
- Seguimientos urgentes.
- Listado completo filtrable.

Plan de pruebas y comprobaciones:

- Aplicar filtros y contrastar resultados con base de datos.
- Comprobar que leads sin contacto solo muestra ultimo_contacto NULL.
- Ver resumen por usuario en admin.
- Navegar desde el listado a detalle.

7.8 Fase 7 · Mejora visual, modo oscuro y objetivos

Funcionalidades desarrolladas:

- Integración progresiva de Tailwind.
- Modo oscuro.
- Sistema global de toast apilable.

- Footer global y política de privacidad.
- Objetivos del mes en dashboard y toast de felicitación por lead ganado.

Plan de pruebas y comprobaciones:

- Cambiar de claro a oscuro en distintas vistas.
- Verificar que los toasts se apilan y desaparecen correctamente.
- Comprobar objetivo del mes y semáforo.
- Cambiar un lead a Ganado y verificar notificación.
- Cambiarlo a Perdido y comprobar `lead_score = 0`.

8. Información para el cliente

8.1 Requisitos físicos y de hardware

Elemento	Requisito recomendado
Equipo cliente	Ordenador de escritorio o portátil con 4 GB de RAM o más.
Procesador	CPU de 2 núcleos o superior.
Disco	Al menos 500 MB libres para proyecto, dependencias y base de datos local.
Red	No imprescindible para uso local; recomendable si se clona desde GitHub o se descargan dependencias.
Pantalla	Resolución mínima recomendada: 1366x768; óptimo en 1920x1080.

8.2 Requisitos software

Componente	Uso
Sistema operativo	Windows 10/11. Compatible también con Linux o macOS con ajustes menores.
Servidor web	Apache incluido en XAMPP.
PHP	Versión compatible con el proyecto PHP MVC y extensiones MySQLi.
Base de datos	MySQL o MariaDB.
Navegador	Chrome, Edge o Firefox modernos.
Composer	Autoload PSR-4 de clases PHP.
Node.js + npm	Opcional para recompilar Tailwind CSS; no necesario para usar la app ya construida.
XAMPP	Entorno recomendado de desarrollo e instalación local.

8.3 Manual de instalación, configuración y administración

Procedimiento recomendado para una instalación local en XAMPP:

1. Copiar la carpeta del proyecto dentro de htdocs de XAMPP.
2. Iniciar Apache y MySQL desde el panel de control de XAMPP.
3. Crear la base de datos importando database/migrations/pipeline_crm.sql en phpMyAdmin.
4. Revisar lib/db_credentials.php y ajustar host, usuario, contraseña y nombre de base de datos si fuera necesario.
5. Ejecutar composer dump-autoload en la raíz del proyecto si el autoload no estuviera generado.
6. Si se van a modificar estilos Tailwind, ejecutar npm install y luego npm run tw:build.
7. Abrir en navegador la URL local del proyecto, por ejemplo: `http://localhost/.../public/`
8. Comprobar que la portada carga y que /login funciona correctamente.

Administración básica del sistema:

- Los usuarios iniciales se crean mediante el script SQL de instalación.
- El rol admin puede eliminar leads y visualizar bloques globales del dashboard.
- Las tablas objetivos_mensuales y notificaciones_app se crean automáticamente al instanciar LeadModel, por lo que conviene acceder al dashboard o al panel tras la instalación para garantizar su existencia en una base limpia.
- El archivo database/views/50_lead.sql puede usarse para poblar la aplicación con datos de prueba.

8.4 Manual de usuario

Uso general para un usuario final del CRM:

1. Entrar en la aplicación mediante la ruta /login e introducir credenciales válidas.
2. Desde la portada pública, usar /contacto para registrar leads externos.
3. En el panel, revisar los leads agrupados por estado y cambiar de etapa cuando avance el proceso comercial.
4. Abrir la ficha de un lead para ver datos, añadir actividades y consultar el histórico.
5. Usar el tablero Kanban si se prefiere una gestión visual por columnas.
6. Crear tareas desde la vista Tareas o dejar que se generen automáticamente cuando un lead entre en Objeciones.
7. Consultar el dashboard para revisar métricas, objetivos del mes, embudo comercial, objeciones y seguimientos urgentes.
8. Usar la vista Listado para localizar leads mediante filtros más generales.
9. Cerrar sesión desde el enlace correspondiente cuando termine el trabajo.

Diferencias de rol:

- El usuario ventas trabaja con su operativa diaria y ve principalmente sus datos asignados.
- El usuario admin dispone de una visión global del sistema y de permisos adicionales, como el borrado de leads y el acceso a resúmenes comparativos por usuario.

9. Bibliografía y fuentes de información

Fuentes técnicas de referencia recomendadas para el proyecto:

- Manuales: <https://www.w3schools.com/>
- Manual oficial de PHP: <https://www.php.net/docs.php>
- Documentación de MySQL y MySQLi: <https://dev.mysql.com/doc/>
- Documentación de Tailwind CSS: <https://tailwindcss.com/docs>
- Componentes Tailwind: <https://flowbite.com/docs>
- MDN Web Docs para HTML, CSS y JavaScript: <https://developer.mozilla.org/>
- Documentación de XAMPP: <https://www.apachefriends.org/>

Conclusión final: PipelineDesk llega a su versión final como una aplicación web académica completa, coherente con la arquitectura MVC y con un alcance funcional suficiente para demostrar captación, seguimiento, automatización básica, visualización analítica y mejora progresiva de la experiencia de usuario.